EEE 304, LAB EC1

The objective of this sequence of laboratory exercises is to explore the use of microcontrollers for the implementation of typical problems in Signals and Systems. MATLAB is also used to as a tool to allow rapid prototyping through its real-time code generation features.

1. Bill of Materials (BOM)

In this laboratory experiment, you will build and test a simple Digital filter on an embedded platform. The idea is to generate a few tones, filter them and observe the output by listening to the sound generated by the tones on a speaker/headphone.

In order to carry out this experiment you will be required to obtain the following:

Table 1: List of parts to be purchased

Item	Model	Quantity
Arduino	Due	1
Micro USB cable	Any android phone charger	1
Headphones	Any headphone used for listening to music. Preferably one that has 20-40 Ohm impedance.	1
Crocodile clips (optional)	Only two clips are needed	2
Digital Multimeter (optional)		1
Set of wires	Male to Male Jumper wires	10
Breadboard	Any breadboard	1

You will require at least the quantities mentioned in the table to successfully carry out the experiment. The hyperlinks attached to the **item names** are for example only and you can purchase these items from anywhere you like. In addition to the official Arduino site (www.Arduino.cc), typical vendors that carry these products are Amazon, Mouser, Digikey, Newegg, etc.

2. Introduction to Arduino DUE

The Arduino Due is a microcontroller board based on a 32-bit ARM core microcontroller (Atmel SAM3X8E ARM Cortex-M3 CPU). It has 54 digital input/output pins (of which 12 can be used as PWM outputs), 12 analog inputs, 4 UARTs (hardware serial ports), an 84 MHz clock, 2 DAC (digital to analog), 2 TWI, a power jack, an SPI header, a JTAG header, a reset button and an erase button. Unlike other Arduino boards it operates at 3.3V. It can be powered using a micro USB port (programming port) or AC-DC adapter. The 12 analog inputs of the Arduino Due have a voltage range of 0-3.3V. It can support

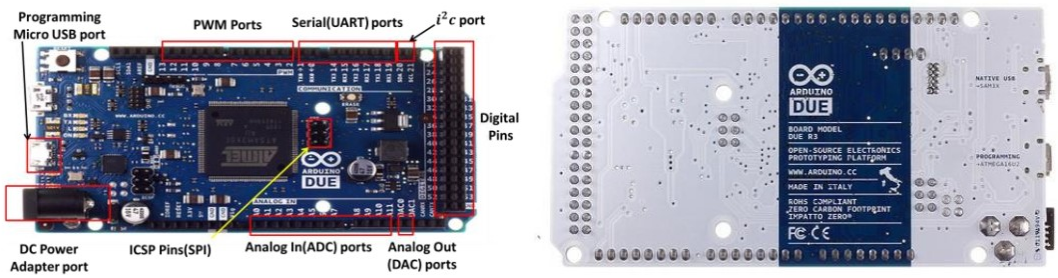
a resolution of 12 bits on each ADC channel. On the other hand, the two DAC channels have a voltage range of

$$1/6 \times (3.3V) \text{ to } 5/6 \times (3.3V)$$

You must keep this limitation in mind while doing experiments with the DAC on the Arduino DUE. The resolution of the DAC is again 12 bits on each channel.

The Due has 4 serial ports (0-3). Serial port 0 is directly connected to the USB programming port and any data can be passed down or read from the Due using that link and any terminal program on the computer (the Arduino IDE has such a program for convenience). All the serial ports operate at 3.3V as high and 0V as low.

All the necessary pinouts of the Arduino Due are shown in Fig. 1. For more details, please visit https://www.arduino.cc/en/Main/arduinoBoardDue.



(a) Top view of Arduino Due (b) Bottom view of Arduino Due

Figure 1: Top and bottom view of the Arduino Due.

2.1 Software Preparation: Installing Toolboxes

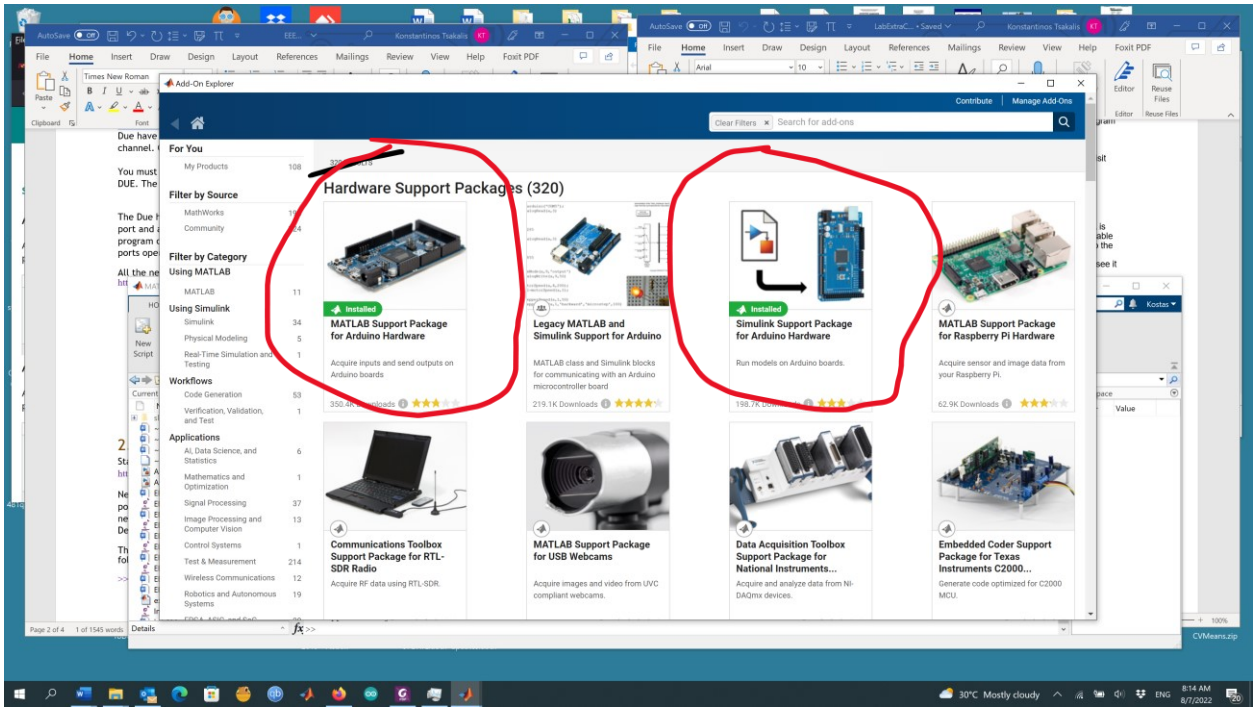
Start by downloading the Arduino IDE software from:
http://arduino.cc/en/Main/OldSoftwareReleases and follow the instructions.

Next, plug in the micro USB end of a cable to the programming port (the one closest to the power jack) and the other end of the cable to the PC. Install drivers for the board, as necessary. Once the drivers are installed, you should see it listed under a COM port inside Device Manager in Windows.

The next step is to install the Simulink Package. In the MATLAB command prompt, type the following and hit enter.

```
>> supportPackageInstaller
```

You will see the installer screen as follows. Click and install the MATLAB and Simulink Support Packages for Arduino Hardware (2 separate packages). The installation takes a few minutes and asks for permissions and MATLAB account (create one if you do not have).



2.2 Setting up a model to run on Arduino DUE

Start by creating a new Simulink Model. Then, under "Tools", select "Run on Target Hardware" and click on "Prepare to run". A new window will appear where you will select "Arduino DUE". The window will change and bring up new menus. Under "Hardware Board Settings", select "Serial port properties". Change baud rate for all serial ports to its highest value, i.e. 115200.

Note: For 2023 and newer Matlab/Simulink versions, adjust the commands as follows:

Hardware -> Run On Board -> External
Then after that is selected...
Hardware -> Prepare -> Hardware Settings

The last bit is to setup the solver parameters for the model. You may use Arduino_test.slx as a test file. On the left side of the same window, select "solver" and configure this page as shown below. Notice that the sinewave generator has a sample time set as the variable TSI (use “Apply” after the edit, will require to click on the warning and resolve the parameter once it is set at the Command Window) and the smallest sample time was also set to TSI (because it is the only one here). The execution time is set to inf (the model keeps running).

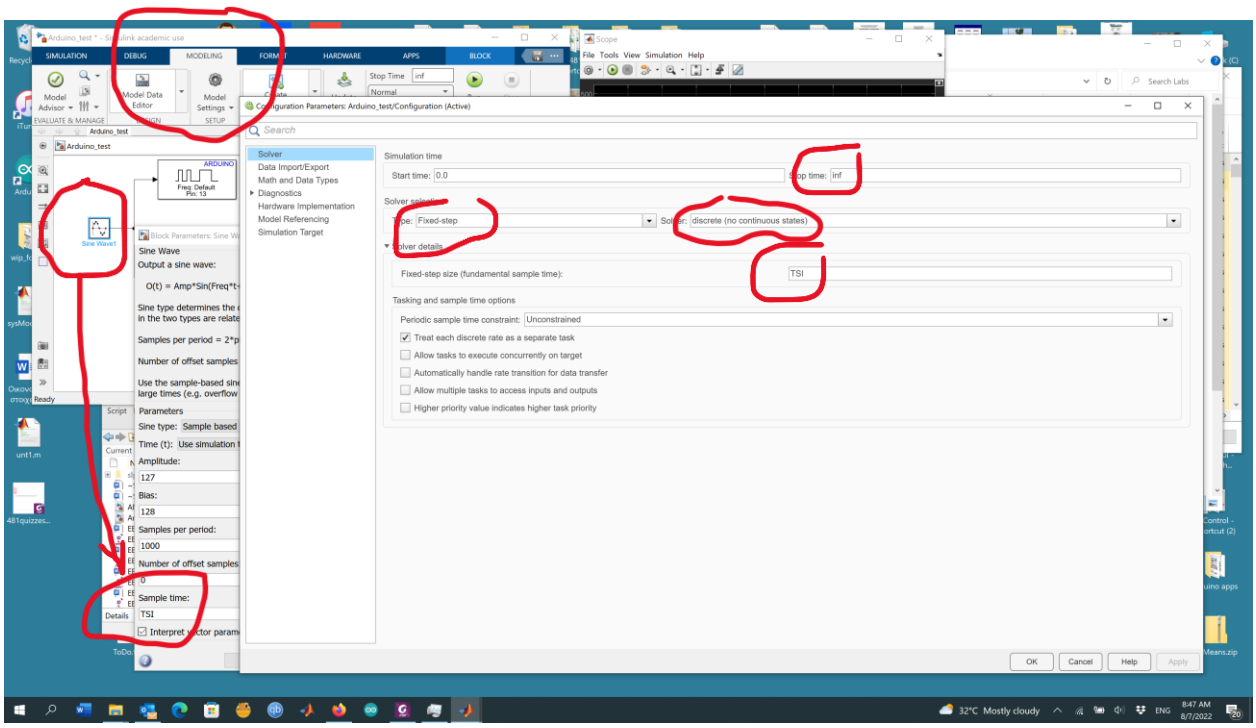
For the configuration screen go to MODELING >> Model Settings >> Model Settings (CTRL-E), in the Solver tab. For all our experiments, and a common safe practice with microcontroller hardware, is to use “Fixed Step” Type and “discrete (no continuous states)” Solver.

At this point, you can connect the Arduino Due to the computer using the USB to micro-USB cable. If Windows asks to install drivers, go through the prompts and install the

drivers. To ensure that the drivers were correctly installed, right click on windows "Start" and select "Device Manager". There under "Ports (COM & LPT)" you should see an item listed as "Arduino Due Programming Port (COMX)", where x is a number e.g., COM3. If this shows up, the Arduino is installed and ready.

At this point, the Simulink Model is ready for code to be built and downloaded to an Arduino DUE.

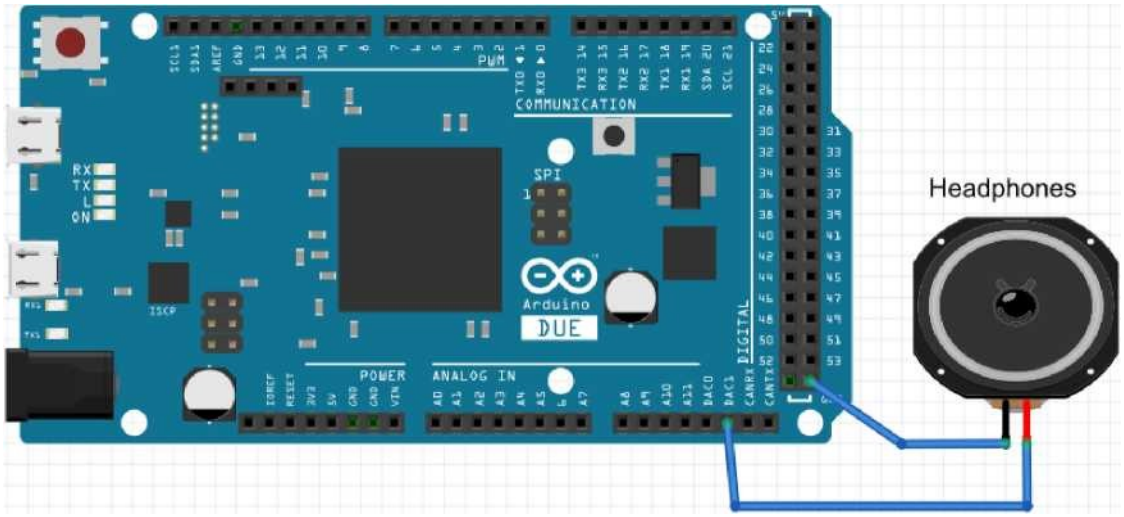
Note: Setting up Arduino after installing the addons is sufficient where you only need to edit model to Fixed Step as shown below.



3. Lab Procedure

3.1 Building the circuit

Using the parts purchased, connect the circuit as shown in the following figure. If you are unsure of the pin labels or for any details, you may look up datasheet for the parts online.



Before connection the headphone to the DUE, it is a good idea to measure the impedance. Typical four pin connectors on stereo headphones have connections as shown in the diagram to the right. If your headphone has only three terminals then you probably do not have volume controls on them. Connect an ohmmeter (from a digital voltmeter) to the GND and Right or Left channel and measure the impedance. Any number between 20 - 40 Ohms is acceptable. **Do not use a speaker with lower impedance because it will burn out the DAC channel on the DUE.** If you have purchased the headphone from the AMAZON link, you will see that the link specifies the headphone has about 32Ohm impedance. (For an actual application, one would need to connect the DAC signal to an audio amplifier, either a single chip or a circuit including different stages, depending on the desired output power.)

In order to connect the headphone to the DUE, you may use two crocodile clips to connect to the headphone and have wires connect from the clips to the GND and DAC1 pin of the Arduino DUE. Alternatively, a make-shift measure would be to wind thin wires around the headphone connectors and insert them to the DUE. Be sure to connect GND of headphone to GND of DUE and any left or right channel of the headphone to the DAC1 pin.

3.2 Building the Simulink Test Model

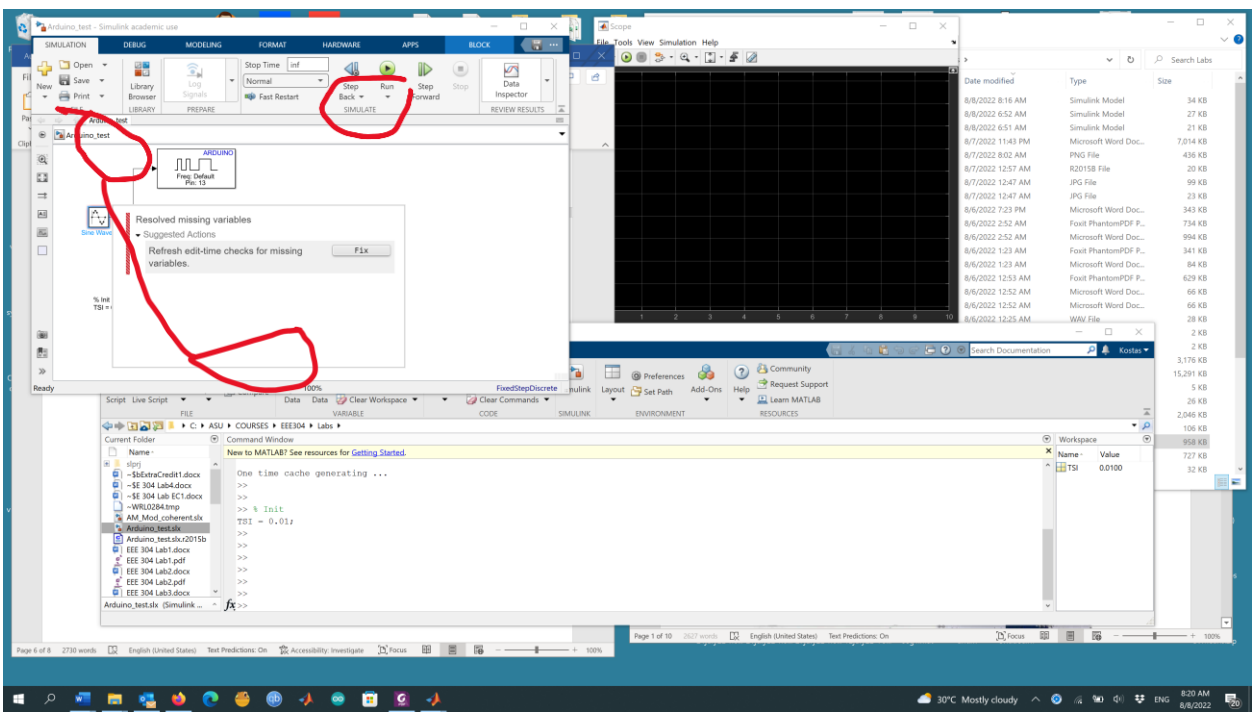
The Simulink model needed to perform this exercise will be provided to you. You do not need to build anything by yourself. Let us start with a simple test model. In your working directory, download and save the "**Arduino_test.slx**" model file, then open it.

> You can open MATLAB, then Simulink by clicking on the button, or typing >>Simulink in the Command Window, or by double-clicking on an .slx file. The last one works if the MATLAB-association is with the version you want to use; it has the advantage that it opens MATLAB with the default folder of the file. Note that MATLAB takes a long time to start and initialize, and so does Simulink. You will know it is done if you hit return in the Command Window and get a new line prompt >>.

Click on MODE and select "External" mode. The Simulink platform allows two modes of compiling code for Arduinos and other target hardware.

- External mode - The model runs on the target hardware but Simulink is constantly downloading and uploading data to and from the board. With this method, you can display and log signals as well as change certain parameters in the model. The disadvantage of this method is that high sampling rates are not supported. Also, the model will not run if the Arduino loses power and its memory.
- **Note: External Mode moved to HARDWARE tab - Run on Board option in the latest versions of MATLAB**
- Normal Mode (Deploy to Hardware) - In this mode the code is built and downloaded onto flash permanently. No message passing occurs between the Arduino and Simulink, so no data display or parameter changing is possible. Very high sampling rates are supported under this mode and the code is permanently in the flash memory so the Arduino can be used when it is not connected to the computer.

For the first test case, we will be using external mode. You can click on PREPARE to set parameters (usually done only once) and specify any initialization parameters in the Command Window. A good practice is to type any initialization commands as a comment in the .slx file so they are saved with it. Once any model parameters have been initialized, look for any red-shadowed blocks that have an error. Click on the block, then the "!" to get info, then on "Fix it" to refresh the model.



With the external mode is selected in the HARDWARE>MODE tab, and Arduino DUE being selected as our Hardware Board, go to DEPLOY to compile and download and start the model.

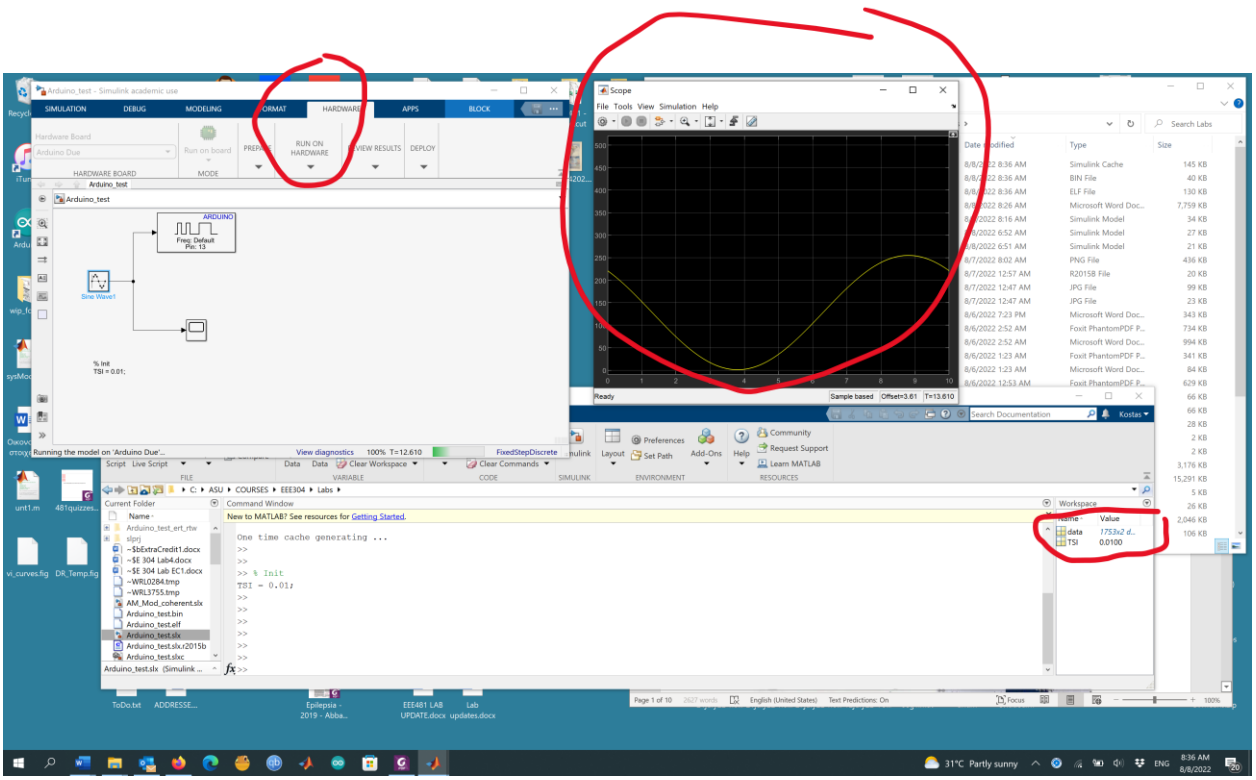
While the model is running, the brightness of the onboard LED on the Arduino DUE should go between 0 and 100% in a sinusoidal timing pattern; this is due to the source

being sinusoidal. Look for the LED next to the USB programming port. There are four LEDs named **RX**, **TX**, **L** and **ON**. The **L** LED is the one we used here which is connected to PWM pin 13.

When you run the model, the sine block generates a sinusoid between 0 and 255. The reason behind this is that the PWM output of the Arduino can accept numbers between 0 and 255 (8- bits) that represents an RMS voltage between 0-3.3V on the pin. That voltage causes the **L** LED to glow at different intensities.

Go to HARDWARE>>RunOnHardware tab and hit the "Monitor and Tune" button. It will rebuild the model and start displaying on the scope the output of the Sinewave block. When you click to Stop the execution the "data" variable will be in the Workspace. This variable has saved time in its first column and the value of the sinusoid in its second column. To plot this data, enter the following command in MATLAB

```
>> plot(data(:,1),data(:,2))
```



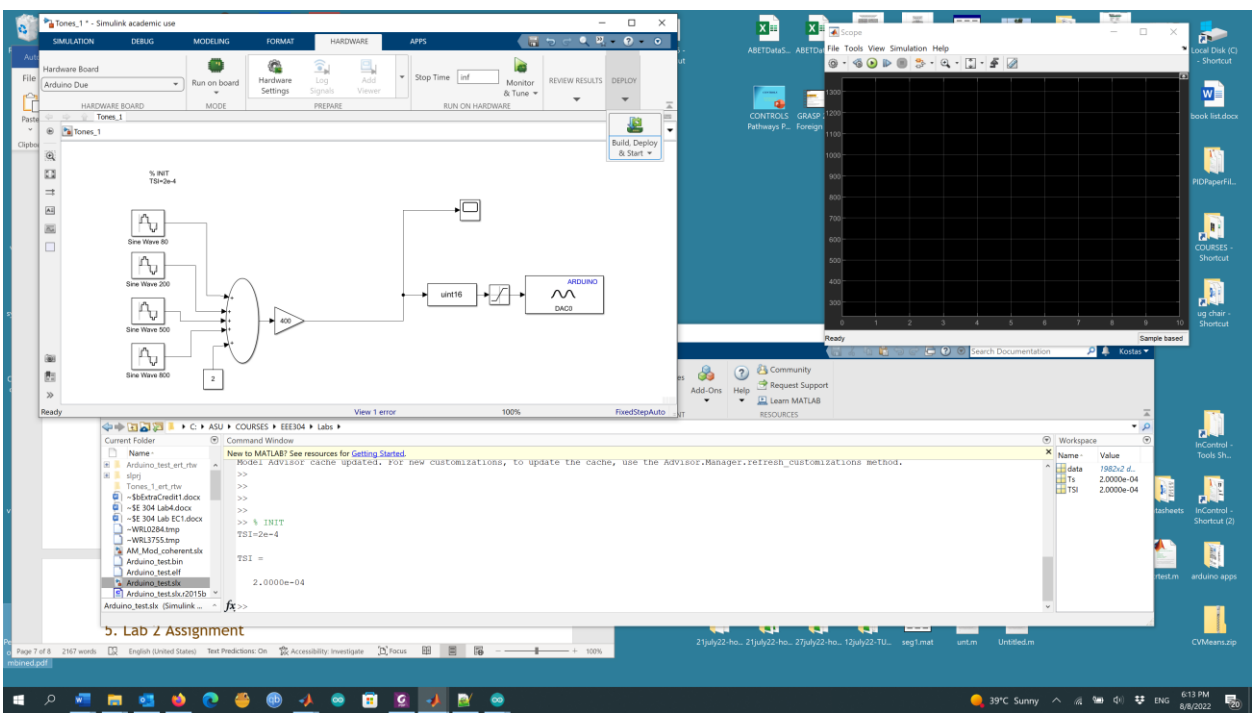
3.3 Building the Simulink Experiment Model

Let us start by looking at what the code does in this Simulink experiment. Four sine waves are generated with frequencies 80, 200, 500 and 800 Hz and amplitude 0.5 peak for all. Since the Arduino DUE DAC can only output positive voltages. The sinusoids are shifted by 2 upwards first. The gain of 400 is applied to modify the output numbers from between 0-4 to about 0-1600. Note that the DAC block in Simulink takes value between 0-4095 since the DUE DAC is 12 bits. This number is converted by the DAC in the DUE to a range of 0.55V - 2.75V as mentioned earlier in this manual. Instead of converting the sinusoid to a range of 0-4095, we chose 0-1600. The reason behind this

is that although the DAC can go up to 2.75V, it cannot deliver enough current. When the speaker with impedance 30 Ohm is connected, the DAC gets loaded and the maximum voltage it can output before it starts clipping is about 1.2V. 0-1600 range in the DAC output ensures the signal stays small and does not get clipped.

Open "**Tones_1.slx**" model file. Make sure your Arduino is connected to the PC via the USB cable. Then click on "Deploy to Hardware". This will compile code and download it onto the Arduino. Remember to initialize the sampling time. Once the process is complete, if the headphone is connected correctly to the DAC1 pin you should be able to hear a tone.

Note: Sometimes you need to move the clip to different parts of the headphone ground to make it work.



3.4 Filtering the Sound

Open and "Deploy to Hardware" the model file "**Tones_2.slx**" as shown in Fig. 12. In this case, we have applied a number of different filters to the same sum of sinusoids generated in the last section. The filters can be activated by grounding either one or both of the Digital pins 7 and 8. For this model, the output is on ADC0 and the parameters are initialized in the workspace by the model. In a dialog box, clicking on the right v-dots can create a variable to which an initial value can be assigned, and subsequently edited.

The figure below shows the case when both pin 7 and 8 are grounded. By default the Arduino Digital Inputs read 1 (high) when no wire is connected, so by connecting a ground to the digital pin makes it read 0 (low). The actions that the model will perform is summarized in the Table 2 below.

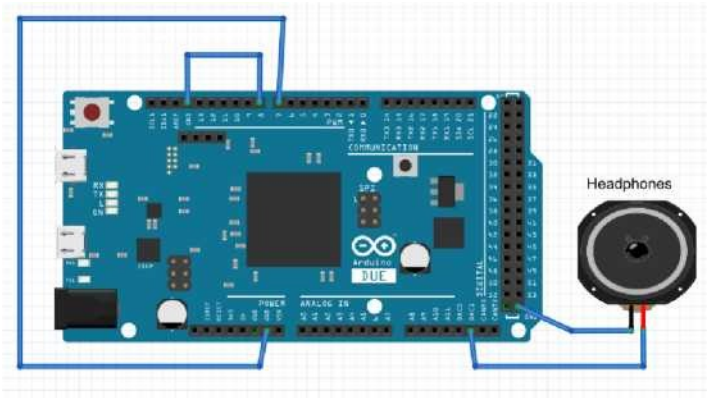


Table 2: Connecting pin 7 and . 8 to activate one of the "If Action Subsystem" blocks

Pin 7	Pin 8	Action
Not Grounded	Not Grounded	The sum of sines will pass through the first "If Action Subsystem" from the top (pass through)
Grounded	Not Grounded	The sum of sines will pass through the second "If Action Subsystem"
Not Grounded	Grounded	The sum of sines will pass through the third "If Action Subsystem"
Grounded	Grounded	The sum of sines will pass through the fourth "If Action Subsystem"

The first "If Action Subsystem" from the top simply allows the signal to pass through without filtering and the sound can be heard on the headphone. This will make the audio sound exactly like you heard in the last section.

For the next three "If Action Subsystem", three different types of filters have been designed. Your task later on will be to identify which is which but for now you will know that **in no particular order**, there is

- A band stop filter to kill all signals so nothing is heard.
- A band pass filter to allow only low frequencies (80, 200Hz).
- A band pass filter to allow only high frequencies (500,800Hz).

Any time you make connections to pin 7 and 8 using Table 1, only one of the "If Action Subsystem" is activated allowing signal to pass through and getting filtered and producing a sound output.

5. Lab EC1 Assignment

#0. Write an overview of your results.

#1. Provide the plot of the "data" variable that you obtained after running the test in Section 3.2.

#2. Does the audio heard on the headphone in the experiment of Section 3.3 sound like it is a single tone or a mixture of frequencies?

#3. As part of your report, connect wires to pin 7 and 8 as per the following table and report on what type of frequencies you can hear. Choose between **high**, **low**, **all** and **none**.

Pin 7	Pin 8	Frequencies heard
Not Grounded	Not Grounded	
Grounded	Not Grounded	
Not Grounded	Grounded	
Grounded	Grounded	

By now you may have found out what type of signal is being filtered. This is exactly how an equalizer filters signals when you listen to music. The last task in this lab is to design some filters

#4. Take a photo of your last hardware set up (Arduino DUE, the headphone and wiring) and attach it to the lab report.

#5. As part of your report, design the following digital filters using the butter command in MATLAB. Provide MATLAB code and Bode plots of these filters. Choose order 4 and sampling rate of 5 kHz.

- A band stop filter to kill all signals so nothing is heard.
- A band pass filter to allow only low frequencies (80, 200Hz).
- A band pass filter to allow only high frequencies (500,800Hz).